

Zatvoon: an open network for on-chain credit

Infrastructure with no opinions, and a way to make idle interest productive

A build note · design stage, nothing deployed

Zatvoon is not a lending product. It is the layer underneath one. It holds no view about which assets are good, which risk parameters are correct, or how much yield anyone should chase. Those are judgments, and judgments belong to people who put their reputation behind them, not to a contract that can never be changed.

This note describes what I am building and how the pieces fit. Nothing here is deployed. I am writing it at the design stage because the ideas are settled enough to be judged, and because the useful criticism arrives before the code, not after.

The problem with having opinions

Most lending protocols are built as a single product with a governance process attached. Somebody decides which assets get listed. Somebody decides the borrowing limits. Somebody decides what counts as an acceptable price source. When conditions change, somebody votes.

This has two costs. The slow one is that listing anything new takes months, so entire categories of asset never get served. The dangerous one is that if a party can change the terms of a live market, then that power exists, and sooner or later someone will want to use it or take it. Historically, the single most productive attack on lending protocols has not been a clever exploit. It has been getting hold of the keys that let you change a market's settings.

Zatvoon removes the power rather than guarding it. Once a market exists, nothing about it can be changed by anyone, including me.

Isolated markets

Anyone can create a market. A market is a pair: one asset people deposit and lend, one asset people post as collateral to borrow against it.

At the moment of creation, the creator chooses everything, and every choice is frozen permanently:

The interest rate curve	frozen
The price source	frozen
The maximum acceptable price age	frozen
The borrowing and liquidation limits	frozen
The share of interest sent to the yield router	frozen

There is no pause, no freeze, and no admin key. I cannot stop a market. I also cannot stop you from withdrawing, and I cannot quietly change the terms you agreed to when you deposited. Those two properties are the same property, and you do not get to keep one without the other.

Markets are isolated. A failure inside one market stays inside it. If a market is badly built, or built deliberately to trap people, the damage is bounded by that market. This matters because permissionless creation guarantees that some markets will be bad ones.

A factory contract mints every market and records it. That registry is how the protocol knows a market genuinely belongs to it, and it is where the few rules that do exist get enforced, before a market is born rather than after.

Changing your mind means creating a new market

Because nothing can be edited, updating a parameter means deploying a new market with the new setting. Users move across if they want to. The old market keeps the exact promise its depositors accepted.

This sounds inconvenient and it is, slightly. It is also the only version where "immutable" means anything. A protocol that lets governance retune a live market is not immutable, it is mutable with extra steps.

The price source: freedom, with one limit

Lending needs prices, and prices come from outside the chain. Zatvoon lets the market creator choose any price source they want. Chainlink, a composed feed, an exchange rate read from another protocol, something they wrote themselves. I have no list of approved oracles and no opinion about which is best.

There is exactly one rule, and it exists because of a specific failure mode.

Price feeds stop. Not as an attack, as routine maintenance: providers retire feeds when they are no longer worth running. If a protocol keeps using the last price it saw, then when the real price moves and the stored one does not, someone can deposit collateral that is no longer worth anything and borrow against the stale number. The market gets emptied.

So every market must declare the maximum age it will accept for a price, and that value cannot exceed seven days. If the price is older than the declared limit, borrowing and liquidation stop. Depositors can still withdraw and borrowers can still repay, because neither of those operations needs to know a price.

Seven days is not a safety guarantee and I will not present it as one. A week-old price is harmless for a stablecoin and catastrophic for a volatile asset, so no single number could be safe everywhere. What the limit does is narrower and worth stating precisely: it guarantees that a dead price feed causes a market to stop working rather than to become exploitable. Failing shut instead of failing open.

For comparison, Morpho's core cannot make this check at all. Its price interface is a single function returning a number with no timestamp attached, so there is nothing to check against. The responsibility sits entirely with whoever wrote the price adapter, and their official adapter does not check either. I think this is a real gap, and closing it costs almost nothing.

Curators: where judgment lives

If the protocol has no opinions, someone has to have them. That is what curators are for.

A curator selects a set of markets and accepts deposits on behalf of people who do not want to evaluate every market themselves. They do the risk work: reading the oracle, checking the parameters, deciding what is worth exposure. They are independent parties with their own names and their own reputations, and that reputation is the entire product.

The important consequence is that Zatvoon does not have a risk profile. Its curators do, and they differ from each other deliberately:

- A conservative curator takes only markets with well-established price sources, cautious parameters, and no interest directed to the yield router at all. Their depositors get a steady, unremarkable return with no exposure beyond the contract code itself.
- An aggressive curator takes newer assets, accepts markets that route a large share of interest outward, and aims for a higher number. Their depositors know what they signed up for.

Both are running on the same infrastructure. Neither is more official than the other. A depositor chooses a curator the way they would choose a fund manager, which is to say by looking at what that person has actually done.

Curators are constrained in the ways that protect depositors rather than in ways that limit judgment. There is no cap on how many markets a curator may include, because a cap would be an arbitrary number frozen into a contract that outlives its own reasoning. Instead: adding a market goes through a delay before it can receive funds, so depositors who dislike the choice can leave first; each market has its own deposit ceiling; and withdrawals draw from markets in a fixed published order, so nobody can jump the queue and leave later withdrawers holding the worse positions.

A market existing on Zatvoon is not an endorsement by Zatvoon. The interface has to make that unmissable, or the first bad market damages everything.

• • •

The part that is actually new

Here is something odd about every lending pool that exists. Borrowers pay interest continuously, second by second. That interest accumulates inside the pool, where it earns exactly the same rate as everything else in the pool. It is capital doing the least imaginative possible thing with itself.

The yield router is the piece that changes this. It takes accumulated interest, and only accumulated interest, and puts it to work in external strategies that pay more. The profit comes back to the depositors of the market it came from.

The word *only* is carrying the weight. No deposit ever leaves its market. The amount working outside can never exceed the interest that has already been earned. This is not a rule someone enforces, it follows from the accounting: the system can only send out what has accrued, and it can only lose what it sent.

The market creator decides the exposure, once

Every market sets a number between zero and one hundred percent: the share of its interest that may be routed outward. It is chosen at creation and frozen with everything else.

Zero is allowed and is a real choice, not a formality. A market at zero never touches the router. Its depositors are exposed to the market's own contract code and nothing else, and that is the honest, literal version of the claim, not a marketing approximation of it.

I originally planned a mandatory minimum of ten percent, so that every market contributed something. I removed it, for two reasons. The first is that it destroyed the cleanest thing about the design: if participation is compulsory, no depositor anywhere can be told their exposure is limited to the contract code. The second is that the minimum bought almost nothing. At ten percent, the interest flows so slowly that the router would take over a decade to reach meaningful size. It was paying the entire cost of the claim for a rounding error of benefit.

There is a better reason to leave it optional. If market creators consistently choose zero, that is the market telling me this piece is not worth its risk, and I want that signal to stay readable rather than suppressing it with a mandate.

How the router decides where to draw from

A second contract groups markets automatically by asset pair. Every market joins its group the moment it is created. This grouping is objective and mechanical, it involves no judgment, and it is deliberately separate from curators. A curator's selections reflect an opinion; a pair grouping is just a fact about which assets a market uses.

The router operates on these groups, and the order matters. Drawing interest out of a market raises its utilisation, so the router draws first from the market with the *lowest* utilisation, then the next lowest, and so on. Returning capital lowers utilisation, so recalls go to the *highest* utilisation market first.

The effect is that the same operation which funds the yield layer also flattens utilisation across markets sharing a pair. Borrowers arriving at a busy market find more liquidity than they otherwise would. The router and the liquidity balancing push in the same direction rather than fighting each other, which was not the original intent but is the reason I am confident the ordering is right.

Sharing one router across all markets rather than running one per market matters more than it sounds. The fixed cost of moving capital does not shrink with the amount moved, so splitting the flow across many markets multiplies the total cost and leaves

interest sitting idle for weeks while each small pot slowly reaches a worthwhile size. One shared router keeps the economics of a single large flow no matter how many markets exist.

When it stops on its own

Sending interest outward reduces the cash sitting in a market, which raises utilisation, which makes borrowing more expensive. Left alone, that loop would feed itself: higher rates mean more interest accruing, which means more to send out.

So the router stops when utilisation reaches a ceiling. This is not a safety valve bolted on afterwards, it is the thing that makes the system stable at all. The deployed amount converges to a level determined entirely by that ceiling, reaches it in roughly two years from an empty start, and then stops growing. No keeper is required, no vote, and no human watching a dashboard.

The same ceiling means the router retreats automatically when borrowers need the liquidity. When utilisation climbs because borrowing demand is high, the router stops sending capital out and begins pulling it back, without anyone deciding to.

Getting capital back

Recall happens in three ways, and none of them runs on a schedule.

The first and most important one lives inside the user's own transaction. If someone withdraws and the market does not have enough cash, the withdrawal itself pulls capital back from the external position, in the same transaction, before completing. The user experiences a normal withdrawal. There is no delay, no queue, and critically no dependence on any bot being alive. If every automated participant in the system vanished permanently, withdrawals would continue working.

The second triggers when utilisation crosses its ceiling, with a deliberate gap between the level that triggers a recall and the level that permits redeployment, so the system does not oscillate and burn money on fees. The third handles the case where an external strategy stops being worth the trouble, and it is the only one allowed to be slow.

Anyone can trigger the routine operations and collect a small reward for the gas. The contract decides how much moves and where it goes; the caller only chooses when. This means there is no privileged operator to rely on and none to compromise.

External strategies and the people who run them

Capital goes to strategies built from a standard template that Zatvoon publishes. A strategist forks it and supplies the strategy logic. The parts that matter for safety, the accounting, the recall path, and the collateral handling, are not overridable in the fork. Governance approves the template and admits individual strategies after audit.

Two design decisions do most of the work here.

Funds never reach a wallet. A strategist receives credit inside the protocol, not tokens. They direct an internal balance along a code path that was fixed before they received anything. There is no transfer out, so there is nothing to run away with. What is being constrained is the set of things the code can do, not the honesty of the person.

Costs are charged, not promised. Every strategy posts collateral. When capital is withdrawn from a strategy, the actual slippage and gas are measured in that same transaction and deducted from the strategist's collateral immediately. A strategist competes on the return net of their own costs, which means they have a direct reason to keep those costs low, and their claims are settled automatically rather than trusted.

The collateral sits in two separate pots: a small one that absorbs routine costs and is topped up from their fees, and a larger locked one that absorbs losses and can only be withdrawn after notice. Keeping them separate matters more than it looks. If routine costs came out of the same pot, they would quietly erode the loss-absorbing layer over a year until it was a fraction of its stated size, with no event and no warning.

Returns are measured by the protocol from its own position, sampled repeatedly across a week, never taken from a rate a strategy advertises. An advertised number can be manufactured in a single block; a measured one cannot, and sampling across the window rather than reading two endpoints is what makes the measurement resistant to a brief manipulation. Collateral raises the ceiling on how much a strategy can receive; it never raises its ranking. Otherwise the best-funded newcomer would outbid the operator with an actual record.

What a depositor is actually exposed to

I want to state this carefully, because the obvious version of the claim is wrong and I would rather be the one to point that out.

If you deposit into a market that routes nothing outward, you are exposed to that market's contract code and nothing else.

If you deposit into a market that does route interest outward, and you have been there since early on, the worst case is that you earn less than you otherwise would. Your claim is your deposit plus everything it has earned, and a complete loss outside cannot exceed the earned portion.

But if you join a market that has been running for a while, you buy in at a price that already includes the accumulated external position. You own a proportional slice of interest that accrued during other people's time, and you have not yet earned anything of your own. A total loss the day after you deposit would come out of your principal.

This is why the deployed amount is capped as a fraction of each market, and why that current fraction has to be displayed before you deposit rather than documented somewhere. The accurate claim is not that your deposit cannot be touched. It is that your exposure is capped at a number you can read before you commit, and that at zero the exposure is genuinely zero.

I am also not going to oversell the size of the benefit. The realistic uplift is a fraction of a percentage point per year, growing slowly as a market ages. It is a structural improvement with an unusually well-bounded downside, not a reason to chase a number.

Zatvoon is being built by one developer, with a co-founder handling the risk mathematics. It is not funded, not launched, and not audited.

This note describes intended design, not a live system. Nothing here is financial advice or an offer of any kind.